

AI Agents: Core concepts

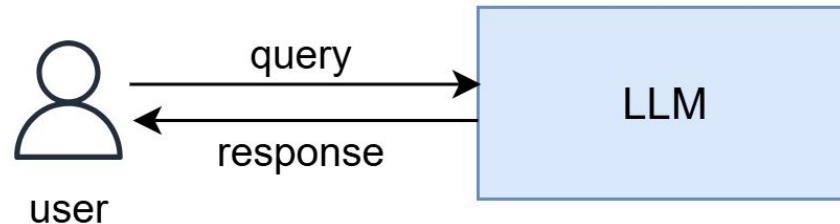
AI Engineering Lab
SoSe 2026

LLMs as conversational interfaces

Transformers
Self-attention
Scaling laws

Pre-training
Post-training
Benchmarking

Multimodality
Mixture of Experts (MoE)
Quantization

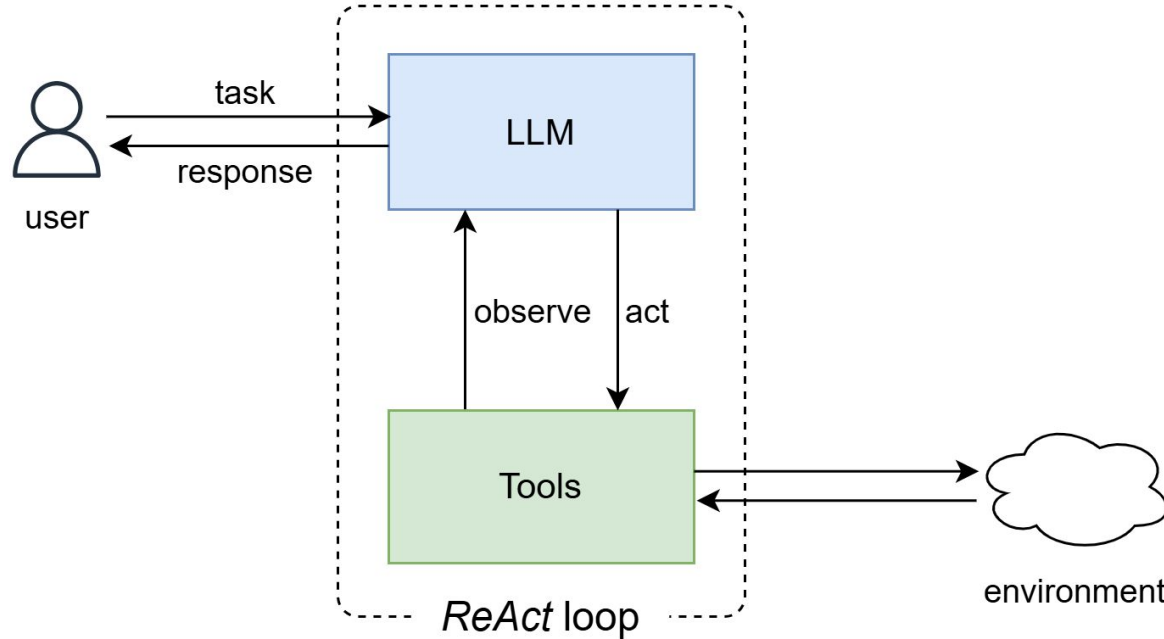


Context window
Prompt engineering
Zero/few-shots learning

Temperature
Hallucinations
Retrieval Augmented Generation (RAG)

Chain-of-Thought (CoT)
Test-time compute
...

AI Agents



Tools:

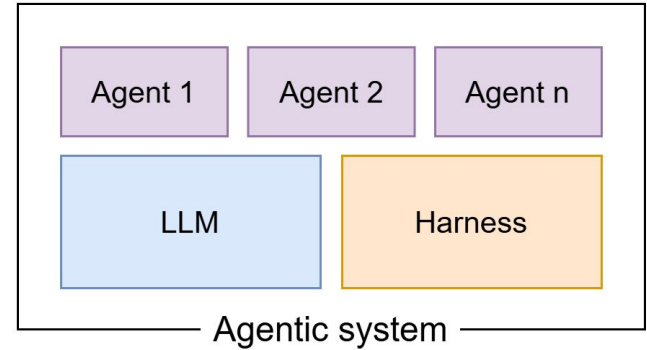
- Search the internet
- Navigate a file system
- Query an API
- Run CLI commands

Tasks:

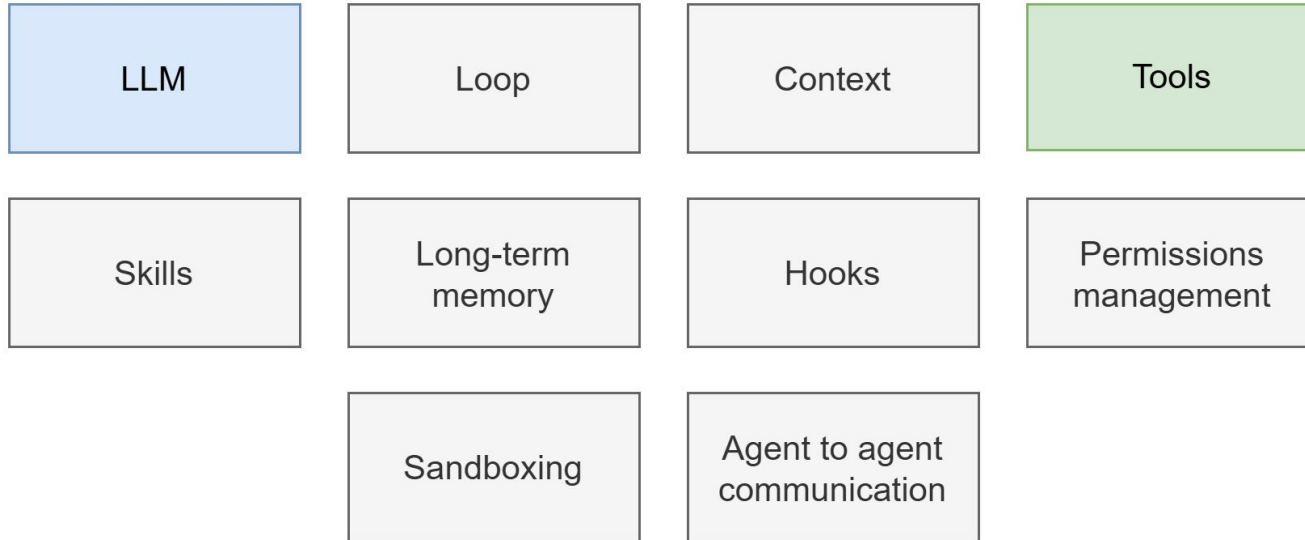
- Booking a flight
- Solving a GitHub issue
- Scheduling a meeting
- Generating leads

Where agents run?

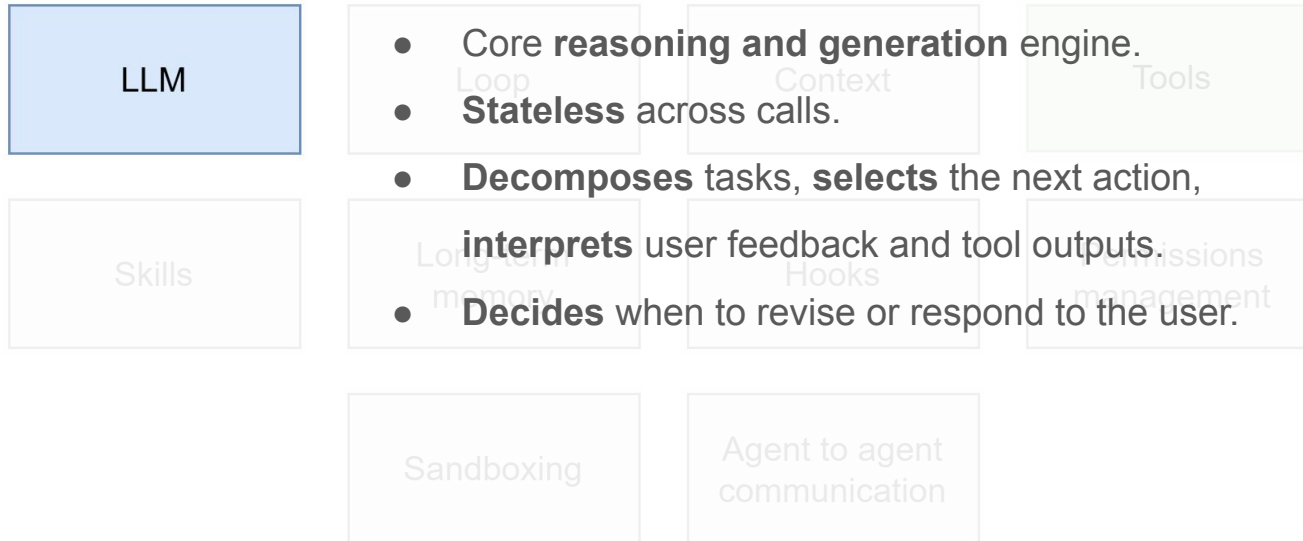
- Agents **vary widely** in terms of:
 - the kind of tasks they can perform
 - the environment in which they act
 - the safety and security implications
 - among other aspects
- **Harness:** infrastructure layer that orchestrates different components.
- Analogy (OOP):
 - *task is an object*
 - *agent is a class*
 - *harness is the platform* (e.g., JVM), with access to the OS (environment)
 - tools are the different *functions* to access OS resources such as CPU, RAM, file system, network, etc.



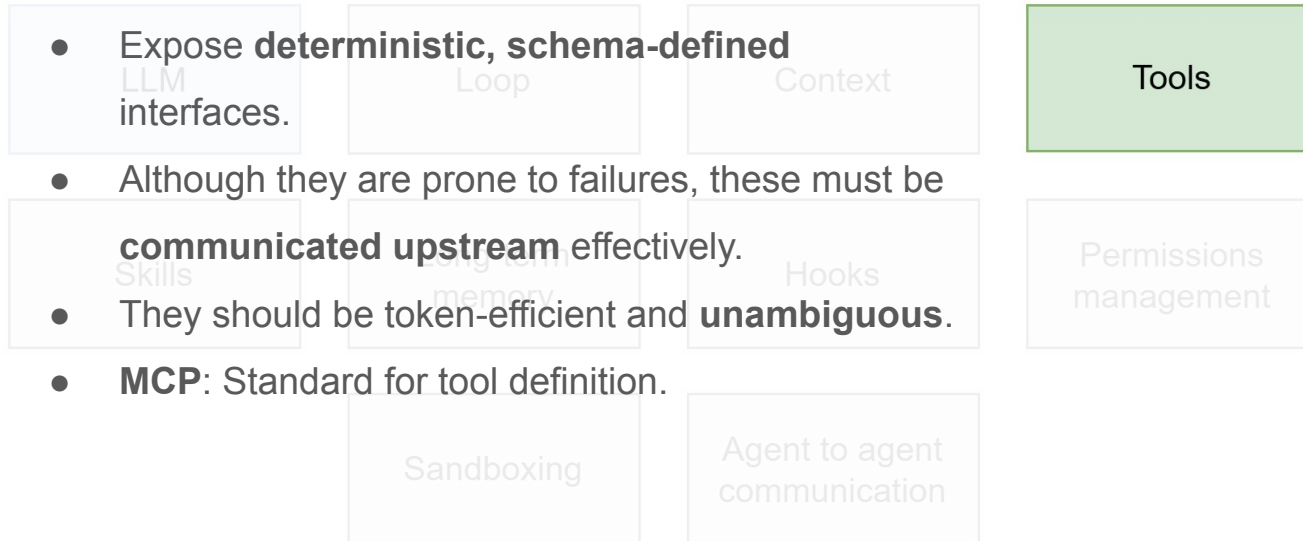
Main components of an agentic system



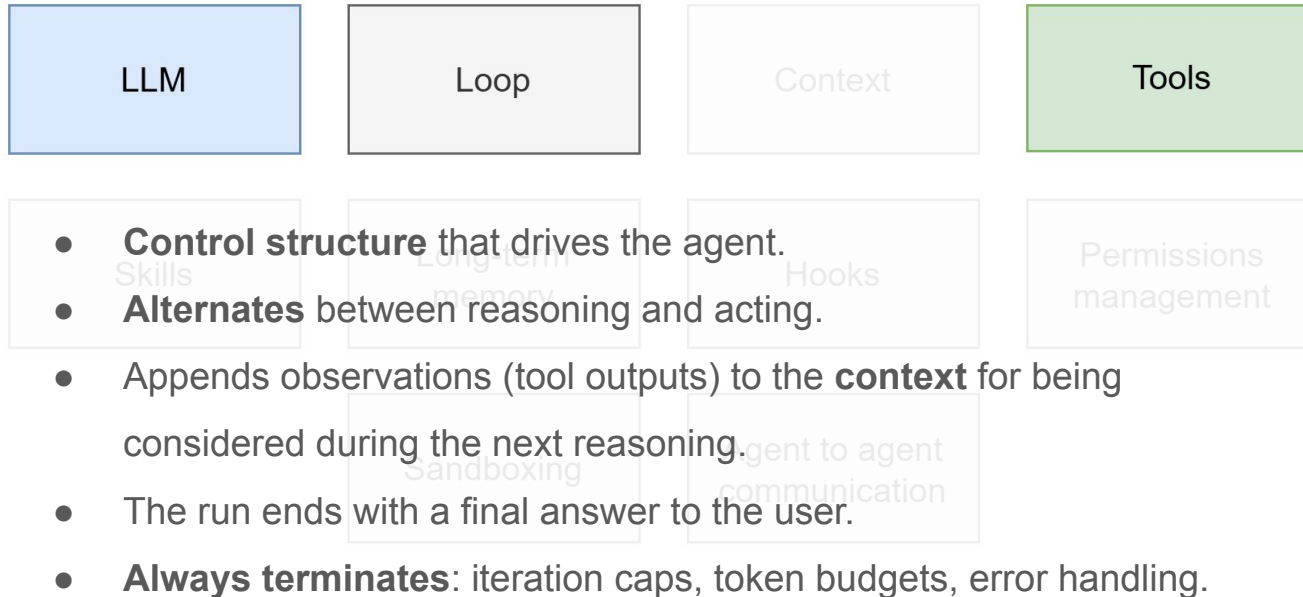
Main components: LLM



Main components: Tools



Main components: Loop

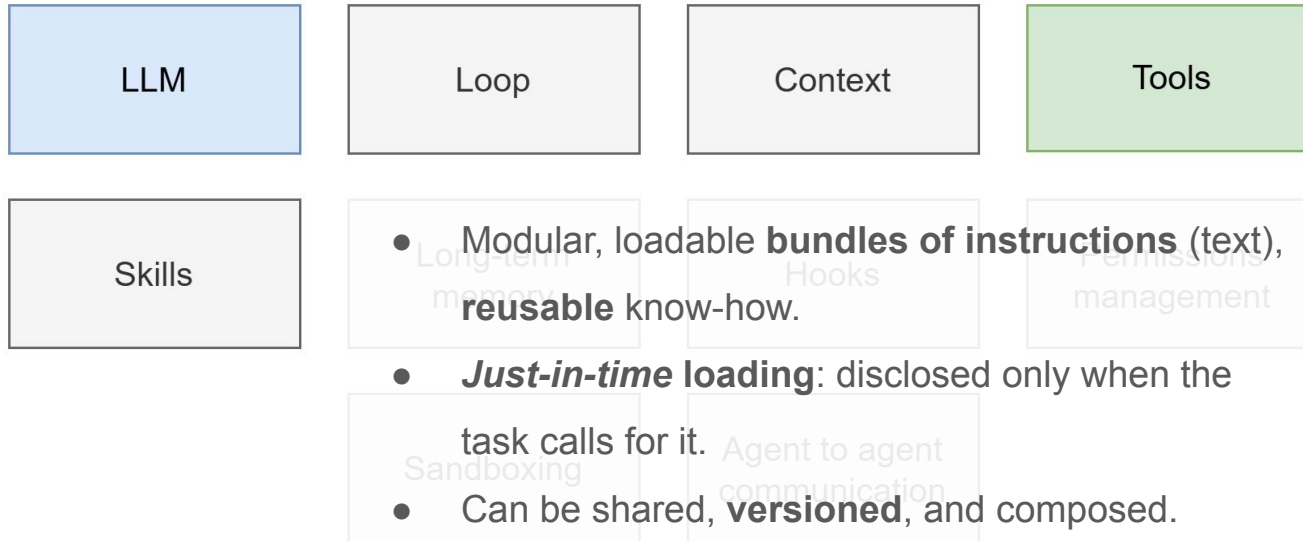


Main components: Context

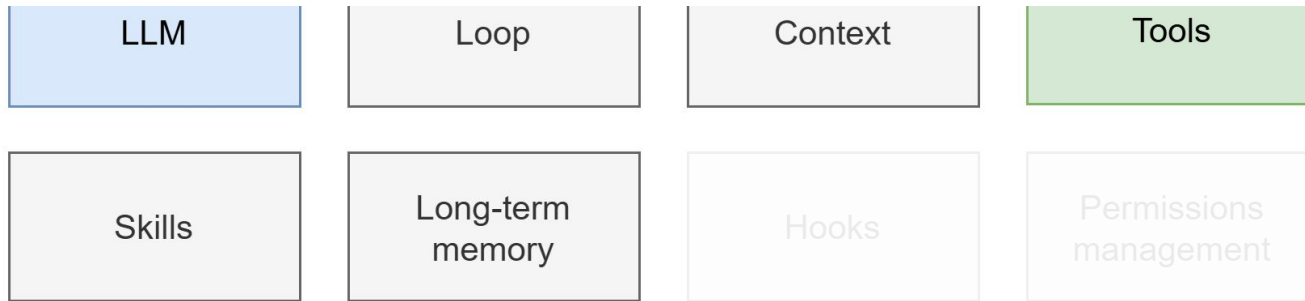


- **Short-term memory**, updated after every loop step.
- Includes system instructions, **conversation history**, tool schemas, and **tool outputs**.
- Conversation grows → **context window** fills → context rot.
- **Context engineering**: seek the smallest set of high-signal tokens that yields the desired behavior (i.e., task finishing).
- **Compaction** and **offloading**.

Main components: Skills

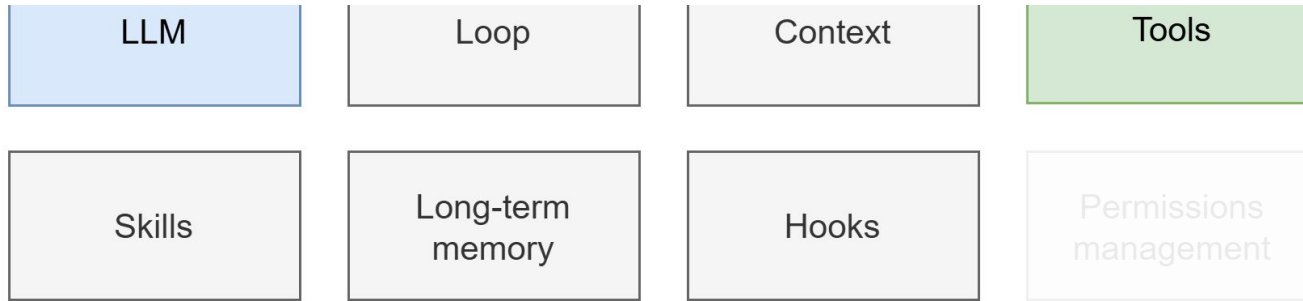


Main components: Long-term memory



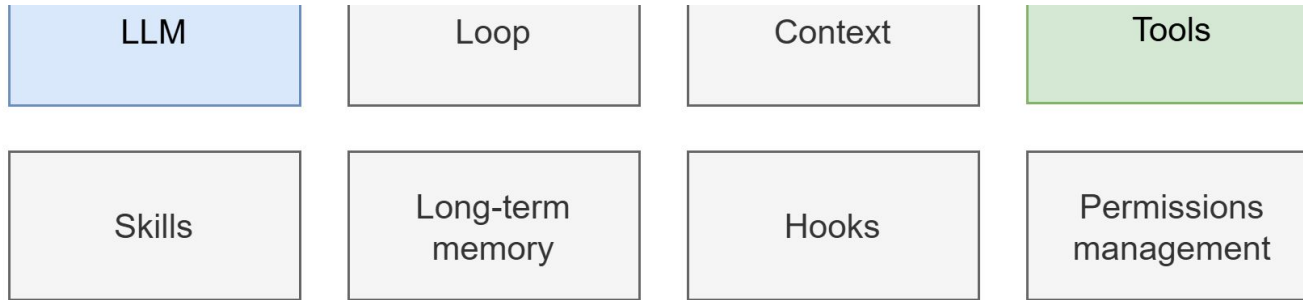
- Persistent information **retained across runs** (e.g., user preferences, learned facts).
- Lives outside the context and is **consulted on demand**.
- On writing: The harness decides **what** is worth persisting and **when**.
- Retrieval combines: dropping **small, high-value** files into the context **up front**, and keeping **lightweight identifiers** to other memory files (*just-in-time* loading).

Main components: Hooks



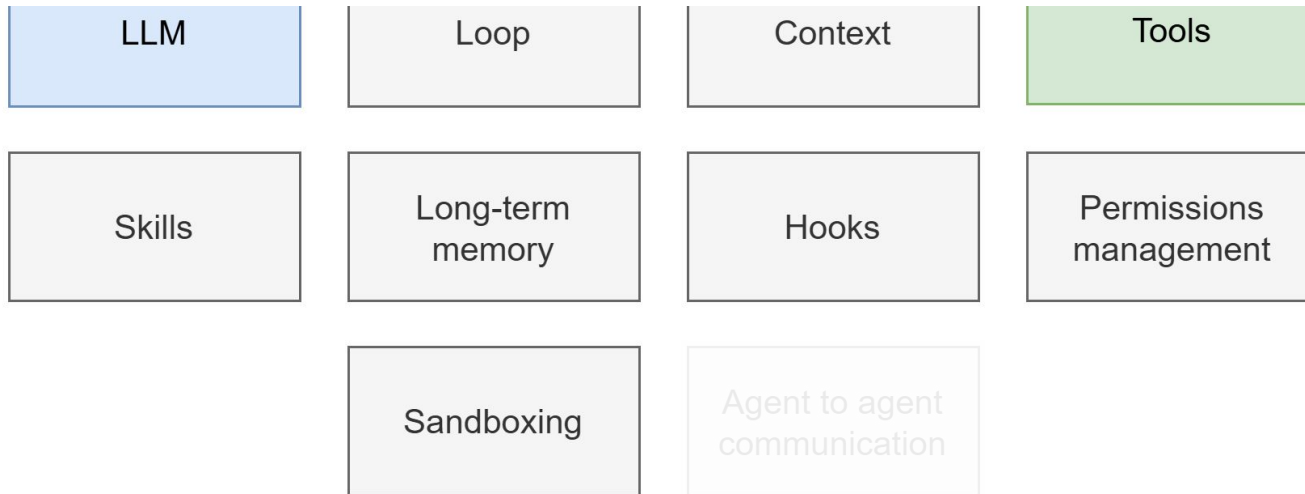
- Custom functions **triggered at specific points** in the agent lifecycle.
- **Pre-use**: validate arguments, enforce permission policies, or block risky commands.
- **Post-use**: sanitize outputs, compact logs, or update memory.
- Natural insertion point for **logging and observability**.

Main components: Permissions management



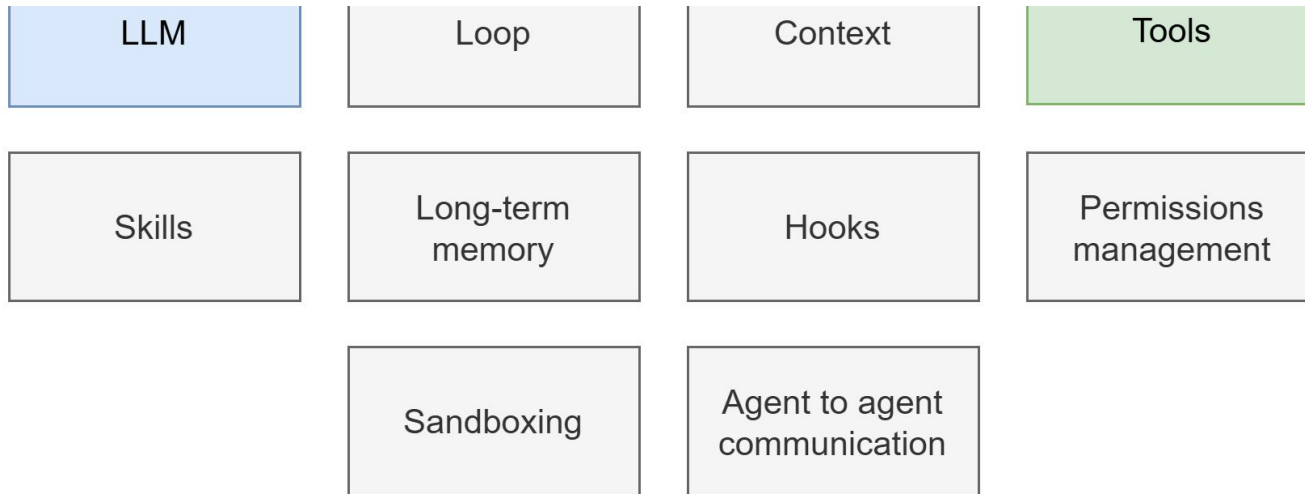
- Policies and controls that determine the actions and resources **an agent may access**.
- Multi-tier model:
 - **read-only** for browsing and inspection.
 - **sandbox-edit** for local changes and test runs.
 - **full-access** for irreversible or outward-facing operations.
- Last tier is guarded by **human-in-the-loop** gates.

Main components: Sandboxing



- Isolation of agent-executed code and tools within a **restricted environment**.
- Different **levels of abstraction**: from OS primitives to a fully separated OS.
- Also improves **reproducibility**: failures reflect a genuine defect rather than env. drifts.

Main components: Agent to agent communication

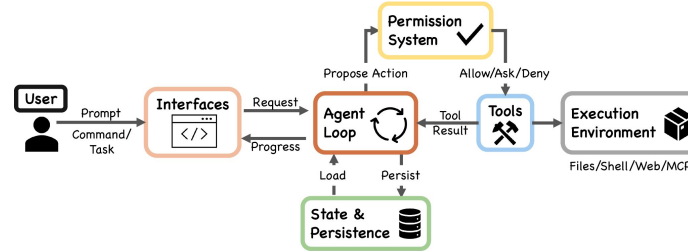


- **Protocols and mechanisms** through which agents coordinate work.
- **Specialized subagents** are a typical case: a main agent delegates a subtask to a specialized one, providing only the relevant info. and a schema of the expected result.
- Intermediate work stays **isolated**, keeping the coordinating **context focused**.

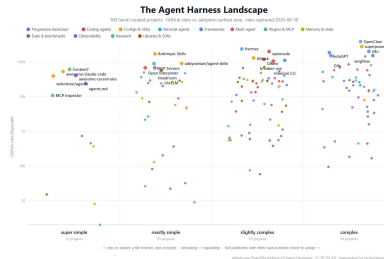
Main components of an agentic system

- Most of these components are **domain-agnostic**: they are defined independently of what an agent running on a harness produces.
- What mainly specializes a harness to a domain are the **concrete tools and skills plugged into it**, along with the sensors used to verify outputs:
 - running a test suite for code,
 - checking layout and consistency properties for a slide deck, or
 - reconciling figures for a report.

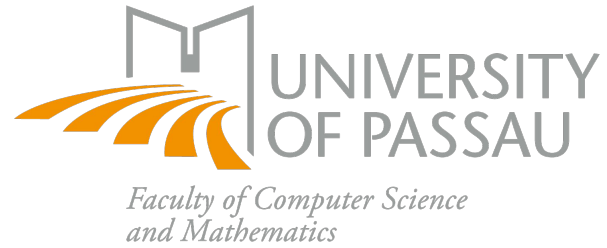
You can draw inspiration from



<https://github.com/VILA-Lab/Dive-into-Claude-Code>



<https://github.com/RyanAlberts/best-of-Agent-Harnesses>



Questions?